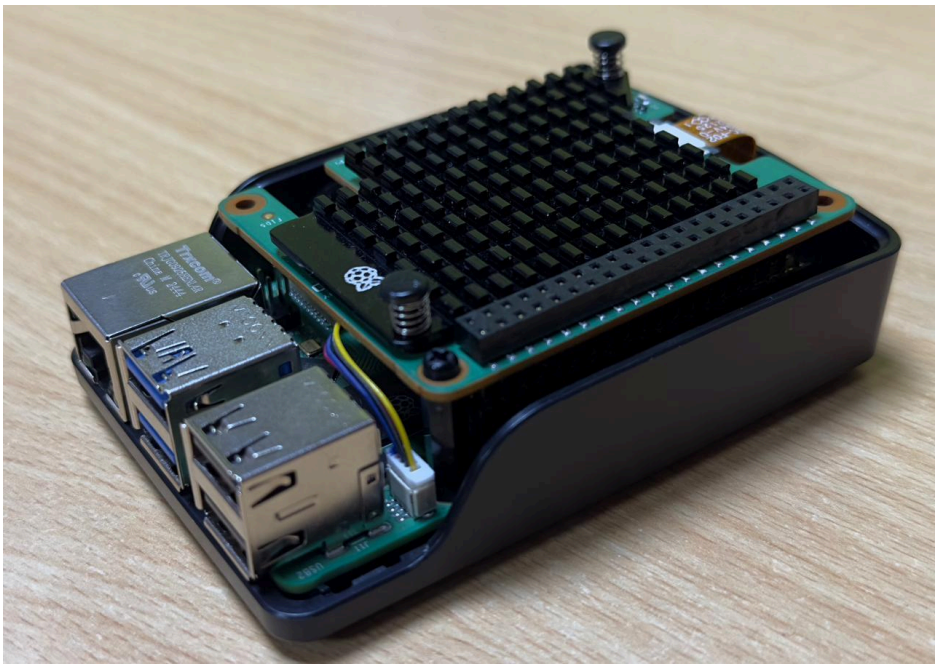
Raspberry Pi 5 — AI HAT+ 2 Setup Guide

Debian Trixie • HailoRT 5.2.0 • hailo-ollama • Open WebUI

ZENLAB-DESE

Compiled by: **Bharadwaj Bhat**

*Under the guidance of **Prof. T. V. Prabhakar***



Overview

The **Raspberry Pi AI HAT+ 2** is an official PCIe add-on board for the Raspberry Pi 5, designed to bring generative AI to the edge. At its core is the **Hailo-10H** neural processing unit (NPU) — a dedicated AI chip that handles neural network inference independently from the host CPU, over a PCIe Gen 3 flex cable.

The key architectural difference from the original AI HAT+ is **8 GB of dedicated on-board LPDDR4X RAM**. Language models need memory to store their weights during inference. On the original AI HAT+, this came directly out of the Pi's 8 GB system RAM. On the AI HAT+ 2, the memory is fully separate, so the Pi's RAM stays free for the operating system and applications.

Running inference on dedicated hardware rather than the CPU delivers practical advantages in lab and edge deployments:

- **Lower inference latency:** the NPU handles token generation without competing with OS processes for CPU time
- **Freed CPU headroom:** all four ARM Cortex-A76 cores stay available for orchestration, I/O, and application logic
- **Consistent throughput:** inference speed does not degrade when background tasks spike
- **No network dependency:** all data stays on the device — essential for privacy-sensitive or air-gapped deployments

Available LLMs at Launch

The following five models are available to install on the AI HAT+ 2 at launch. All are quantised to INT4 and compiled specifically for the Hailo-10H. They cannot be swapped for arbitrary GGUF or ONNX files — only models pre-compiled by Hailo are supported through hailo-ollama.

Model	Parameters	Speciality	Best Used For	Approx. Speed (Hailo-10H)
DeepSeek-R1-Distill	1.5 billion	Chain-of-thought reasoning; internal reasoning trace before final answer	Step-by-step problem solving, maths, logical deduction, multi-step decisions	~6–7 tokens/s; TTFT ~2–4 s
Llama 3.2	1 billion	Lightweight general-purpose language model; good instruction following at a minimal memory footprint	Simple Q&A, summarisation, light chat, quick instruction tasks	~7–8 tokens/s; TTFT ~1–2 s
Qwen2.5-Coder	1.5 billion	Code generation, debugging, and code explanation; trained on a large corpus of programming languages	Writing functions, explaining code snippets, fixing bugs, and code completion	~6–7 tokens/s; TTFT ~2–3 s
Qwen2.5-Instruct	1.5 billion	Instruction-tuned for structured tasks; follows	Summarisation, translation, structured data	~6–7 tokens/s; TTFT ~2–3 s

Model	Parameters	Speciality	Best Used For	Approx. Speed (Hailo-10H)
		formatting and output-style instructions reliably	extraction, document Q&A	
Qwen2	1.5 billion	Strong multilingual support across 29+ languages; competitive general text generation	Non-English language tasks, translation, multilingual chat, cross-language content generation	~6–7 tokens/s; TTFT ~2–3 s

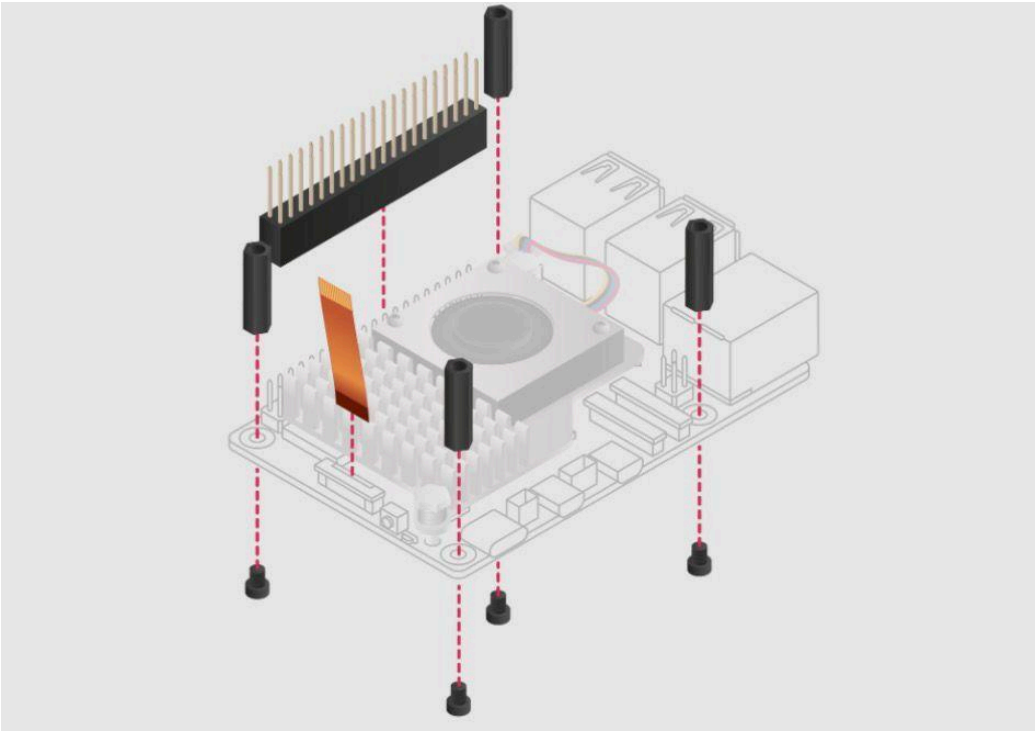
> ***NOTE:*** Speed figures are approximate, measured on the Hailo-10H in independent testing [4]. Actual performance varies with prompt length and context size. **TTFT** = Time to First Token. **tokens/s** refers to generation speed after the first token appears. We have provided the code to check performance.

AI HAT+ vs AI HAT+ 2

The core change is the shift from the Hailo-8/8L (designed for vision workloads) to the Hailo-10H (re-architected for generative AI). The Hailo-10H also uses **INT4 quantisation** rather than INT8, which roughly doubles the effective compute per watt at the cost of slightly reduced numerical precision.

Feature	AI HAT+	AI HAT+ 2
Accelerator chip	Hailo-8L (13 TOPS) or Hailo-8 (26 TOPS) — INT8	Hailo-10H (40 TOPS) — INT4
On-board memory	None — borrows from Pi 5 system RAM	8 GB dedicated LPDDR4X
LLM support	Not supported	Supported (up to ~6B parameters)
VLM support	Not supported	Supported
Primary use cases	Object detection, camera pipelines, robotics	All AI HAT+ workloads plus local LLMs, VLMs, speech-to-text, translation

Hardware Assembly



> **WARNING:** Disconnect the Raspberry Pi 5 from all power sources before touching any hardware. Connecting the PCIe cable under power or static discharge can permanently damage both boards.

The HAT ships with all required hardware in the box: four threaded spacers, four long screws, four short screws, a GPIO stacking header, and a PCIe ribbon cable. A Phillips crosshead screwdriver is needed. A Raspberry Pi Active Cooler is strongly recommended — the Hailo-10H generates meaningful heat during inference.

1. **Power off and unplug the Pi.** Do not just shut down the OS — physically disconnect the power cable.
2. **Attach the four spacers.** Thread the long screws through the mounting holes on the Pi's board and tighten the spacers from below.
3. **Press on the GPIO stacking header.** Align all 40 pins with the GPIO connector and push down firmly and evenly until the header sits flush.
4. **Disconnect the PCIe ribbon cable from the HAT.** Slide the locking tabs outward on both sides of the connector, then pull the cable straight out.
5. **Connect the ribbon cable to the Pi.** Lift the locking tab on the Pi's PCIe flex connector. Insert the cable with the copper contacts facing inward — toward the USB ports. Push the tab back down until it clicks.

6. **Mount the AI HAT+ 2.** Lower the HAT onto the spacers with the components facing up and secure it with the four short screws.
7. **Reconnect the ribbon cable to the HAT.** Open the HAT's PCIe connector tab, insert the cable, and close the tab firmly.

For the full illustrated assembly guide, see the official documentation [1]: https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html

Raspberry Pi OS Setup

Why Debian 13 Trixie is Required?

Debian 13 (Trixie) is a hard requirement, not a recommendation. There are three specific reasons why the previous Bookworm OS (kernel 6.6.x) does not work with the AI HAT+ 2:

1. The hailo-h10-all package refuses to install on kernel 6.6.

The package declares an explicit dependency on BCM2712 kernel headers at version 6.12.x. When `dpkg` checks this on Bookworm (which has `6.6.x+rpt-rpi-2712`), the check fails immediately. The package will not even unpack — you see a dependency error before any driver compilation begins.

2. The Hailo-10H uses a completely different driver stack that was never built for kernel 6.6.

The original AI HAT+ (Hailo-8) used HailoRT 4.x. The Hailo-10H requires HailoRT 5.x, which introduces a rewritten PCIe kernel module (`hailo1x_pci` instead of `hailo_pci`). Hailo only compiled and validated this module against kernel 6.12. Even if forced to install, the DKMS build step would fail because the 5.x driver calls scatter-gather DMA and MSI-X interrupt APIs that changed between 6.6 and 6.12. There is no backport.

3. The BCM2712 PCIe bridge driver changed between kernel 6.6 and 6.12.

In kernel 6.6, the Pi 5's PCIe controller was driven by a downstream RPi-specific driver with known DMA addressing limitations — this is why Bookworm users needed the `pcie-32bit-dma` device tree overlay for anything to work on that bus. In kernel 6.12, the BCM2712 bridge switched to the upstream `pcieport` driver with corrected DMA range descriptors. The Hailo-10H driver's DMA memory allocation calls assume the upstream driver is managing the bridge. On 6.6, those calls either fail silently or allocate memory in a region the hardware cannot address, which is exactly why `lspci` may show the device while `hailortcli scan` returns nothing.

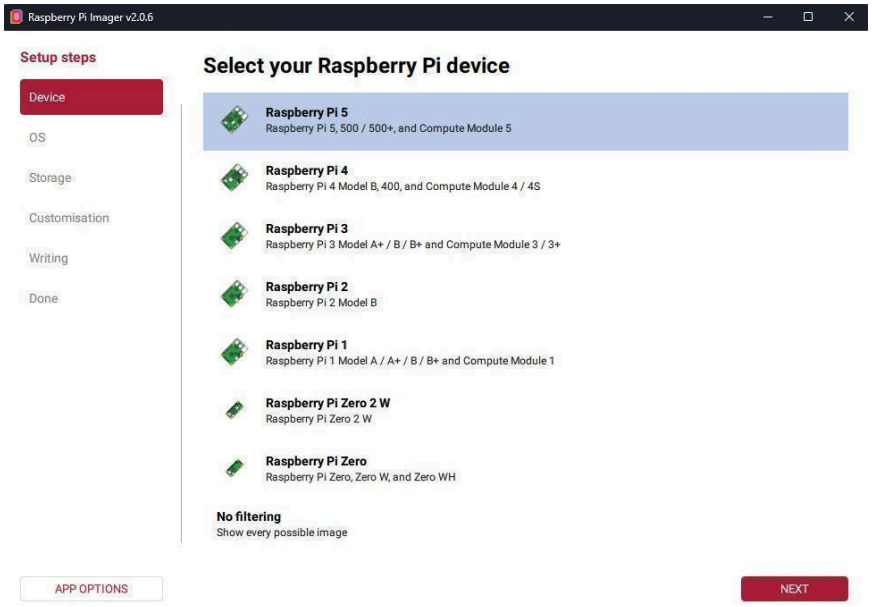
> *Bookworm does not fail because it is old. It fails because the Hailo-10H driver was built against a version of the BCM2712 PCIe stack that only exists in kernel 6.12. Trixie is the first Raspberry Pi OS that ships the right kernel for this hardware to function.*

Beyond the kernel requirement, Trixie also brings:

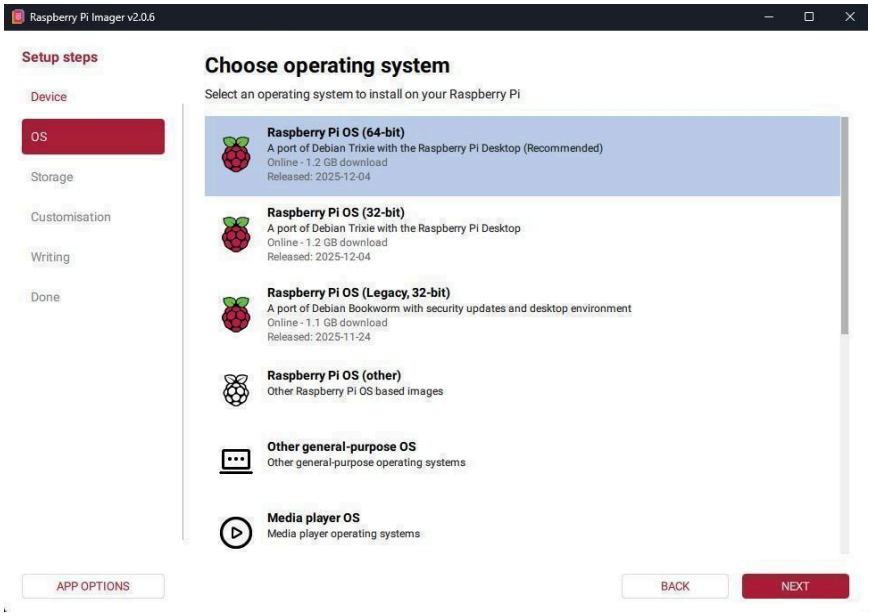
- **Linux kernel 6.12.51-v8:** updated PCIe subsystem with improved interrupt and DMA coalescing
- **Wayland / labwc compositor:** replaces the legacy X11 desktop stack
- **Independent auto-login configuration:** console and desktop auto-login can be controlled separately, useful for headless lab deployments

Flashing the OS with Raspberry Pi Imager

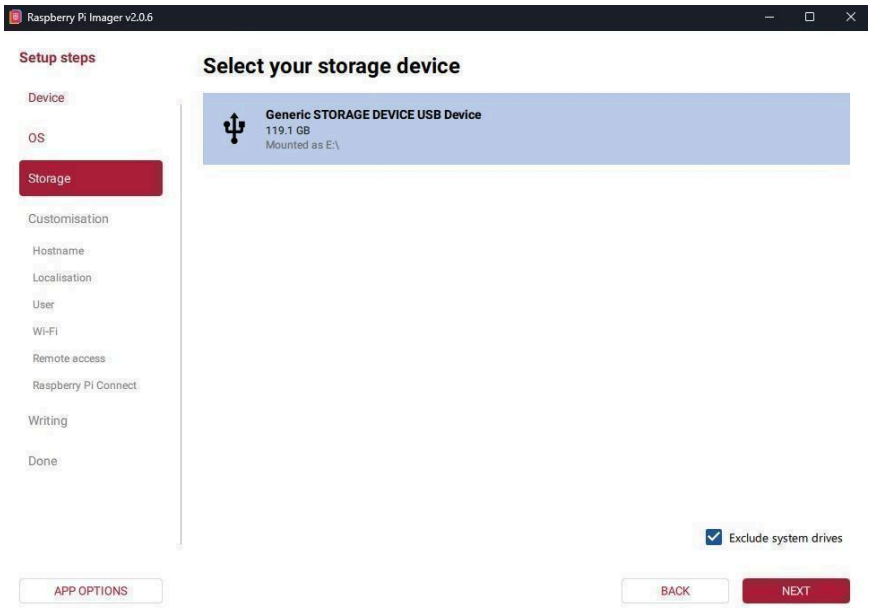
Step 1: Open Raspberry Pi Imager (v1.8.5 or later is recommended). Under **Device**, select **Raspberry Pi 5**.



Step 2: Under **OS**, select **Raspberry Pi OS (64-bit)** based on Debian Trixie. As of early 2026, this is listed under the “Other” submenu in Imager.

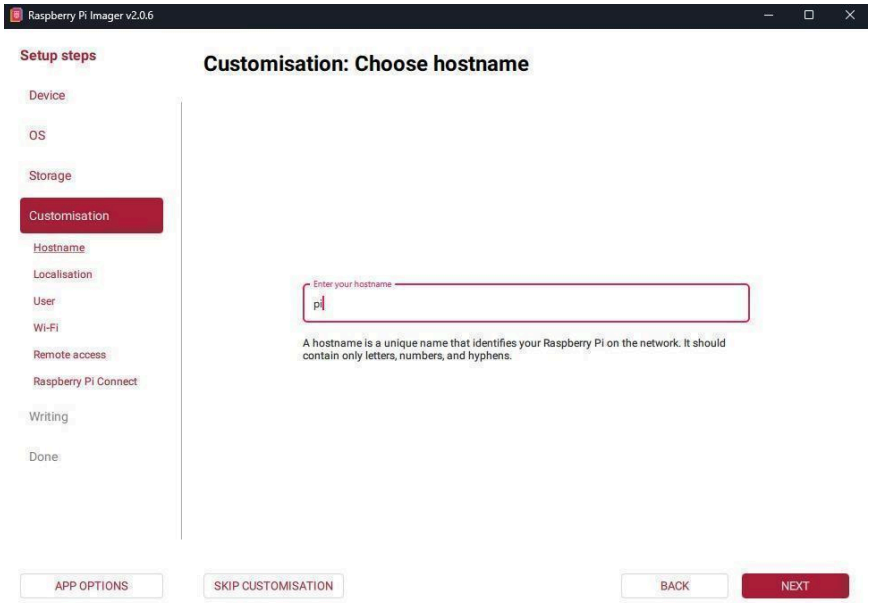


Step 3: Under **Storage**, select your target media. A **64 GB or larger card** is strongly recommended — the HailoRT packages, Docker image, and model weights combined use over 10 GB. A USB 3.0 SSD will give better I/O performance than an SD card for Docker operations. Enable **Exclude system drives** to prevent accidentally overwriting host storage.

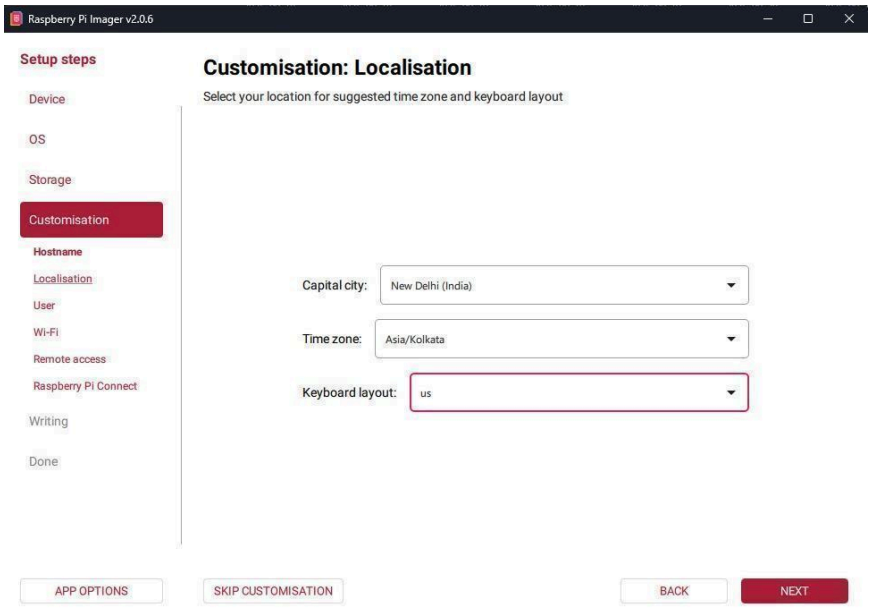


Step 4 (Critical — do not skip): Click **Edit Settings** in the OS Customisation dialogue. This is the only opportunity to configure the Pi before the first boot:

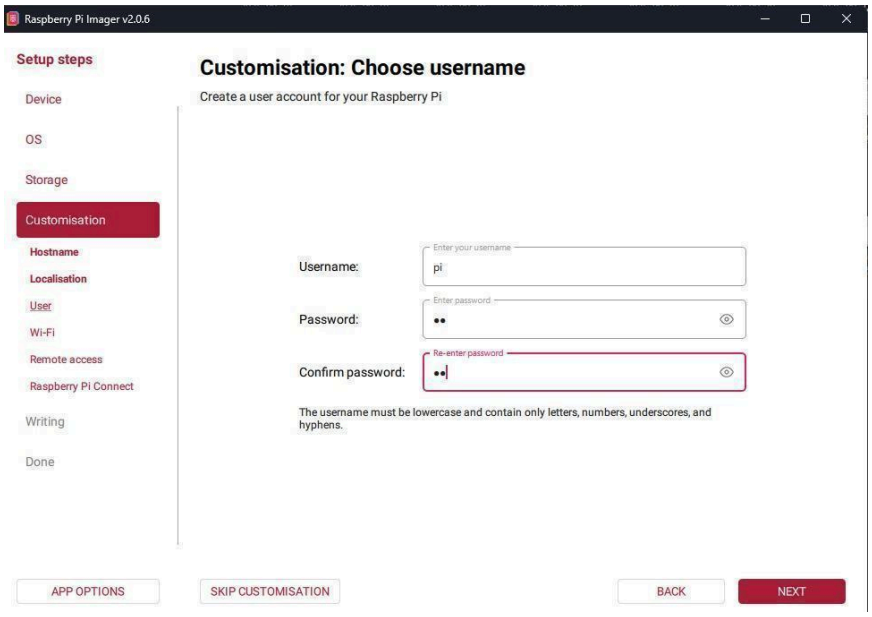
Hostname: Set a recognisable name for the device, for example `rpi5-hailo`.



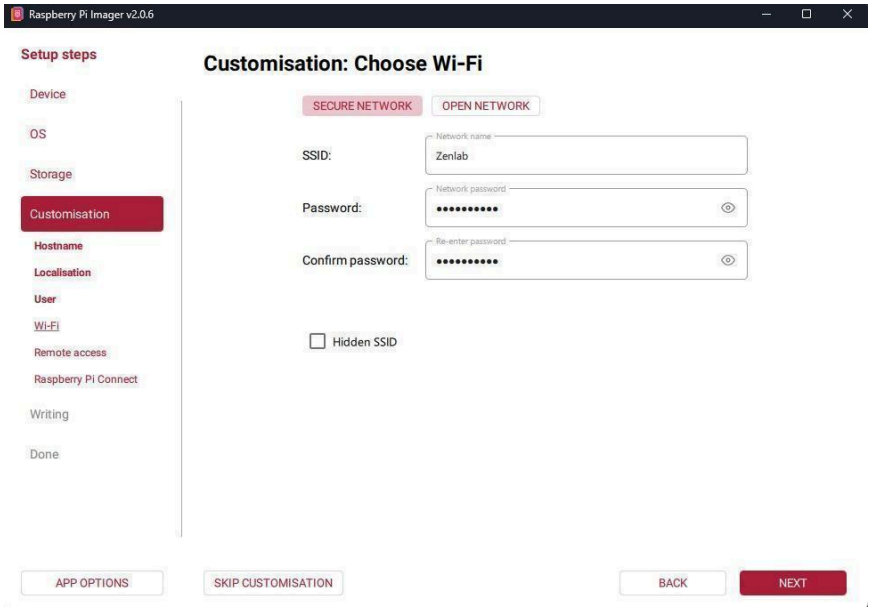
Localisation: Select your region, time zone, and keyboard layout.



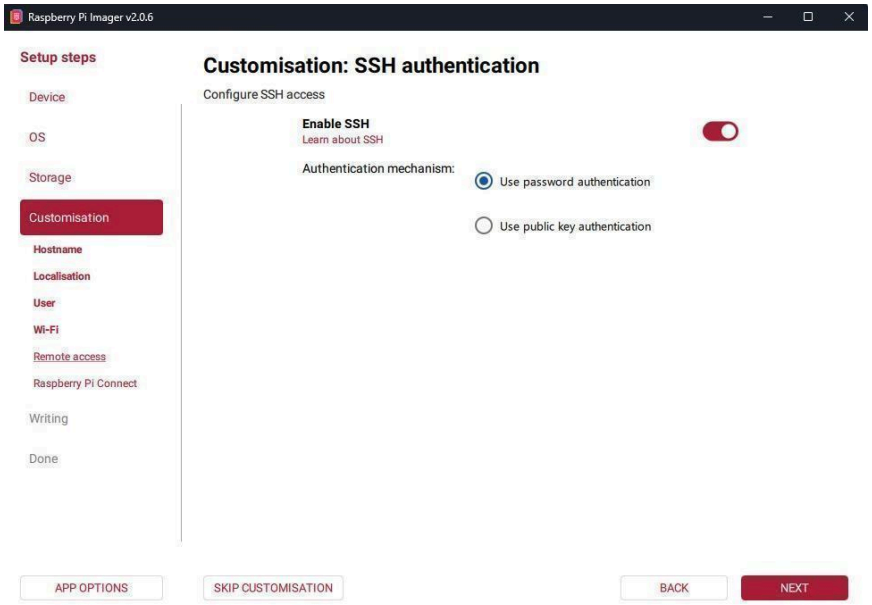
Username and password: Set a secure password for the default user. The default `pi/raspberry` credential is publicly known and should not be used.



Wi-Fi credentials: Enter your network SSID and password if you are not using a wired Ethernet connection.

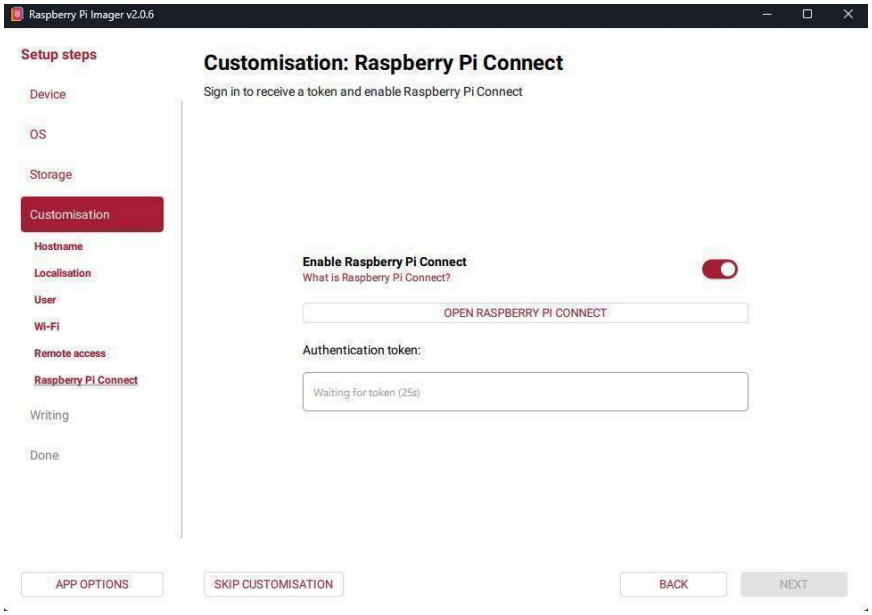


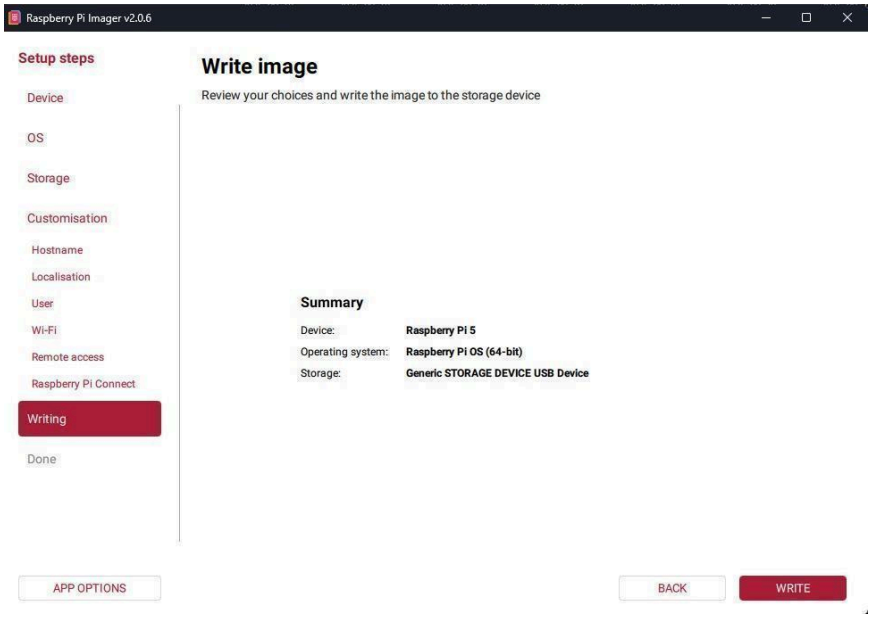
Enable SSH: Tick this option under the Services tab. Without SSH, you will need a keyboard and monitor connected for every interaction with the Pi.



> *Skipping Step 4 means the Pi will boot without SSH access and without a known password, requiring a keyboard and monitor for initial configuration. Alternatively, you can use Raspberry Pi Connect to access the Pi remotely after logging in from a laptop or PC — but this still requires completing the initial setup on screen first.*

Step 5: Click **Write**. The Imager will ask you to confirm, then write and verify the image. This typically takes 3–5 minutes, depending on the card or SSD speed. Once writing is complete, click **Finish** and remove the storage media from the computer.





Step 6: Insert the prepared storage into the Raspberry Pi 5 and power it on. Allow approximately 90 seconds for the first boot to complete.

Initial Raspberry Pi Setup

Before installing any Hailo software, update the system fully. This ensures the package manager has the latest index, all existing packages are at their newest versions, and the correct kernel headers are present for DKMS to compile against.

Update the package list and upgrade all installed packages:

```
sudo apt update && sudo apt full-upgrade -y
```

Reboot after upgrading to apply any kernel updates before installing drivers:

```
sudo reboot now
```

After rebooting, confirm the running kernel version:

```
uname -r
```

> ***NOTE:*** *The output should show 6.12.x-v8 or later. If it still shows 6.6.x, the Trixie image was not selected correctly during flashing — re-flash the SD card with the correct Trixie build.*

Enable the PCIe Gen 3 interface (required for the AI HAT+ 2):

```
sudo raspi-config
```

Navigate to **Advanced Options** → **PCIe Speed** → select **Yes** to enable PCIe Gen 3. Exit and reboot when prompted.

Optionally, verify the PCIe bus is active after reboot:

```
lspci
```

> ***NOTE:*** *The Hailo-10H will appear as a PCIe device in the output even before any Hailo driver is installed. This confirms the PCIe ribbon cable is seated correctly and the Gen 3 mode is active.*

Phase 1 — Install Hailo Drivers

Install `dkms` (Dynamic Kernel Module Support) and the `hailo-h10-all` meta-package. DKMS automatically recompiles the PCIe kernel module when the system kernel is updated, preventing driver breakage after `apt upgrade`. The `hailo-h10-all` package pulls in the PCIe driver, HAT firmware, and all required library dependencies in one step.

> **WARNING:** The correct package name is `hailo-h10-all` — not `hailo-all`. Running "sudo apt install hailo-all" installs the Hailo-8 driver, which targets a completely different chip. It will appear to install successfully but the AI HAT+ 2 will not be detected by `hailortcli`. Always confirm you are installing the h10 variant.

```
sudo apt install dkms
sudo apt install hailo-h10-all
sudo reboot now
```

Phase 2 — Download HailoRT 5.2.0 Packages

The Hailo apt repository currently only serves HailoRT **5.1.1**. Version **5.2.0 is required** for hailo-ollama — version 5.1.1 does not support the Hailo-10H generative AI workloads. The 5.2.0 .deb packages must be downloaded directly from the Hailo Developer Portal.

Why you must use a browser to download — not wget or curl: The Hailo Developer Portal requires you to be logged in before any file download is allowed. When you log in through a browser, the browser stores a login cookie and the download works normally. Command-line tools like `wget` and `curl` do not have this cookie, so the portal sends back a small HTML login-redirect page (~98 KB) instead of the actual package. The download appears to succeed — you get a file with the right name — but `dpkg` will reject it because it is not a valid Debian package. The only reliable way to get the files is to log in through Chromium on the Pi and download from there.

Open Chromium on the Pi and log in at:

```
https://developer.hailo.ai/developer-zone/sw-downloads/
```

After logging in, download these three files. They are saved to ~/Downloads/ by default:

8. hailort_5.2.0_arm64.deb
9. hailort-pcie-driver_5.2.0_all.deb
10. hailo_gen_ai_model_zoo_5.2.0_arm64.deb

After downloading, verify the files are genuine Debian archives and not HTML redirect pages:

```
file ~/Downloads/hailort_5.2.0_arm64.deb
ls -lh ~/Downloads/*.deb
```

> ***NOTE:*** *Expected file sizes: hailort ~5 MB, hailort-pcie-driver ~28 MB, model zoo ~693 KB. If any file shows ~98 KB, it is an HTML redirect page — delete it, confirm you are logged in to the portal, and re-download.*

Phase 3 — Remove Old Hailo Packages and Install 5.2.0

Installing 5.2.0 on top of 5.1.1 without removing the older packages first will produce a dependency conflict. The removal command below strips all relevant 5.1.1 components. The `2>/dev/null` redirect suppresses errors for packages that were never installed — these are safe to ignore.

Step 1 — Remove old Hailo packages:

```
sudo dpkg -r rpicam-apps-hailo-postprocess python3-h10-hailort \
    hailo-h10-all h10-hailort 2>/dev/null
```

Step 2 — Install 5.2.0 in the correct order (PCIe driver first, then runtime, then model zoo):

```
sudo dpkg --install \
    ~/Downloads/hailort-pcie-driver_5.2.0_all.deb \
    ~/Downloads/hailort_5.2.0_arm64.deb \
    ~/Downloads/hailo_gen_ai_model_zoo_5.2.0_arm64.deb

sudo apt-get -f install -y
sudo reboot now
```

Step 3 — After rebooting, verify the Hailo chip is detected:

```
hailortcli scan
hailortcli --version
```

> ***NOTE:*** *Expected output from scan: Device: 0001:01:00.0. Expected version output: 5.2.0. If the device is not listed, check that the PCIe ribbon cable is fully seated at both ends.*

Troubleshooting: h10-hailort-pcie-driver Version Conflict

Some systems have a conflicting `h10-hailort-pcie-driver` package that blocks the 5.2.0 driver install. To diagnose, first check what is installed:

```
dpkg -l | egrep 'hailo|hailort|h10'
```

If `h10-hailort-pcie-driver` is listed, remove it explicitly before reinstalling:

```
sudo apt-get remove --purge -y h10-hailort-pcie-driver
sudo dpkg -i ~/Downloads/hailort-pcie-driver_5.2.0_all.deb
sudo apt-get -f install -y
sudo reboot
```

Phase 4 — Install Docker

Open WebUI requires Python 3.11 or earlier. Debian Trixie ships Python **3.13** as the system interpreter, which is incompatible with several Open WebUI dependencies. Docker provides a clean, isolated container with the correct Python version already included — this is the approach recommended by Open WebUI itself [5].

```
sudo apt install docker.io -y
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

> ***NOTE:*** *newgrp docker activates the group membership for the current shell session only. You must fully log out and log back in for the change to apply automatically in all future sessions.*

Phase 5 — Install and Start hailo-ollama

`Hailo-ollama` is an inference server that exposes an Ollama-compatible REST API on port 8000. When an LLM request arrives, it routes the computation to the Hailo-10H over PCIe rather than executing it on the CPU. Any client that works with a standard Ollama server also works with Hailo-Ollama without modification.

Installation

On Debian Trixie, Python is managed under PEP 668 — the system interpreter is marked as externally managed, so `pip install` will refuse without an explicit override. Use the `--break-system-packages` flag:

```
pip install hailo-ollama --break-system-packages
```

Starting the Inference Server

Run `hailo-ollama` in a dedicated terminal. It will occupy that terminal while running — this is expected. Do not close it.

Terminal 1 — start the inference server:

```
hailo-ollama
```

Terminal 2 — list available models and pull the ones you need:

```
curl --silent http://localhost:8000/hailo/v1/list

curl --silent http://localhost:8000/api/pull \
    -H 'Content-Type: application/json' \
    -d '{"model": "deepseek_r1:1.5b", "stream": true}'

curl --silent http://localhost:8000/api/pull \
    -H 'Content-Type: application/json' \
    -d '{"model": "llama3.2:1b", "stream": true}'

curl --silent http://localhost:8000/api/pull \
    -H 'Content-Type: application/json' \
    -d '{"model": "qwen2.5-coder:1.5b", "stream": true}'

curl --silent http://localhost:8000/api/pull \
    -H 'Content-Type: application/json' \
    -d '{"model": "qwen2.5:1.5b", "stream": true}'

curl --silent http://localhost:8000/api/pull \
    -H 'Content-Type: application/json' \
    -d '{"model": "qwen2:1.5b", "stream": true}'
```

> ***NOTE:*** *Wait for {"status":"success"} after each pull before starting the next. Running concurrent pulls can produce partial or corrupted model files.*

Phase 6 — Install and Launch Open WebUI

`Open WebUI` is a browser-based chat interface that connects to hailo-ollama's API on port 8000. The `--network=host` flag is required so the Docker container can reach hailo-ollama running on the host network stack. Without it, the container is isolated in its own network namespace and cannot connect.

```
docker pull ghcr.io/open-webui/open-webui:main

docker run -d \
    -e OLLAMA_BASE_URL=http://127.0.0.1:8000 \
    -v open-webui:/app/backend/data \
    --name open-webui \
    --network=host \
    --restart always \
    ghcr.io/open-webui/open-webui:main
```

Open Chromium and navigate to (use `http`, not `https`):

```
http://127.0.0.1:8080
```

> ***NOTE:*** *The --restart always policy means Docker will start the Open WebUI container automatically on every boot. However, hailo-ollama is not running at that point, so Open WebUI will show a connection error until hailo-ollama is launched. This is expected — use the Everyday Use command below to start everything in the correct order.*

Everyday Use — After Every Reboot

After rebooting, run this single command to bring everything back up in the correct order. It starts the Docker daemon, restarts the Open WebUI container, and launches hailo-ollama if it is not already running. The terminal will be held by hailo-ollama — leave it open.

```
sudo systemctl start docker && docker start open-webui && pgrep hailo-ollama || hailo-ollama
```

Then navigate to:

```
http://127.0.0.1:8080
```

Quick Reference

Command	Purpose
`hailortcli scan`	Verify the Hailo chip is detected on the PCIe bus
`hailortcli --version`	Check the installed HailoRT version (should be 5.2.0)
`hailo-ollama`	Start the inference server (keep this terminal open)
`docker ps`	Confirm the Open WebUI container is running
`curl ...hailo/v1/list`	List all downloaded Hailo-compatible models
`http://127.0.0.1:8080`	Access Open WebUI in the browser (http, not https)
`vcgencmd measure_temp`	Read the Raspberry Pi CPU temperature
`free -h`	Show current system memory usage
`docker logs open-webui`	Inspect Open WebUI startup logs for errors
`sudo systemctl status docker`	Check whether the Docker service is running

Using Models from Python

`Hailo-ollama` exposes the same REST API as a standard Ollama server. The official `ollama` Python package can connect to it by passing a custom `host` parameter — no other code changes are needed. Inference always runs on the Hailo-10H chip regardless of how the request is sent.

Install the client:

```
pip install ollama --break-system-packages
```

Example usage:

```
import ollama

client = ollama.Client(host="http://127.0.0.1:8000")

response = client.chat(
    model="qwen2.5-coder:1.5b",
    messages=[{"role": "user", "content": "Your prompt here"}]
)
```

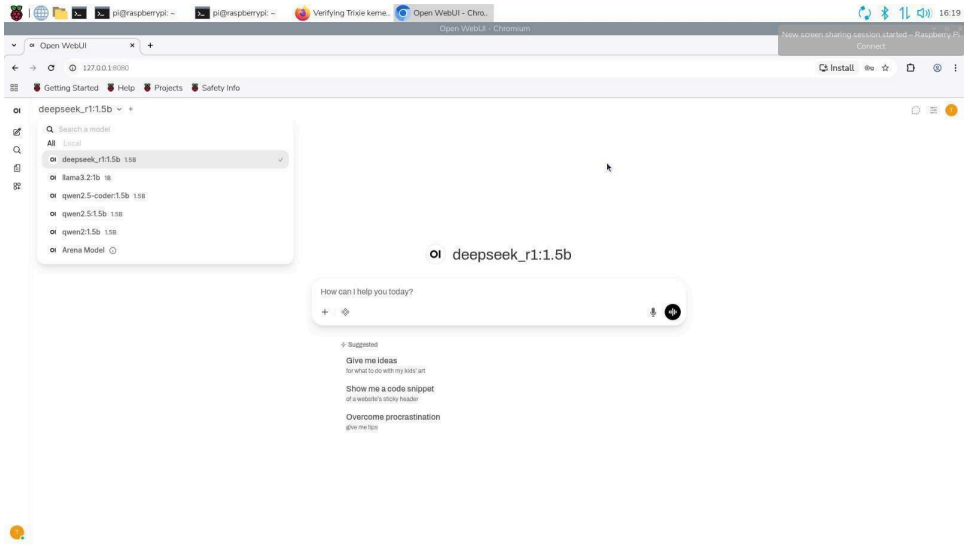
```
# The ollama library returns an object, not a dictionary.
# Use attribute access --- not response["message"]["content"].
print(response.message.content)
```

> ***NOTE:*** *Works from any Python environment (virtualenv, conda, system Python) on the same Pi, as long as hailo-ollama is running. The --break-system-packages flag is required on Debian Trixie due to PEP 668.*

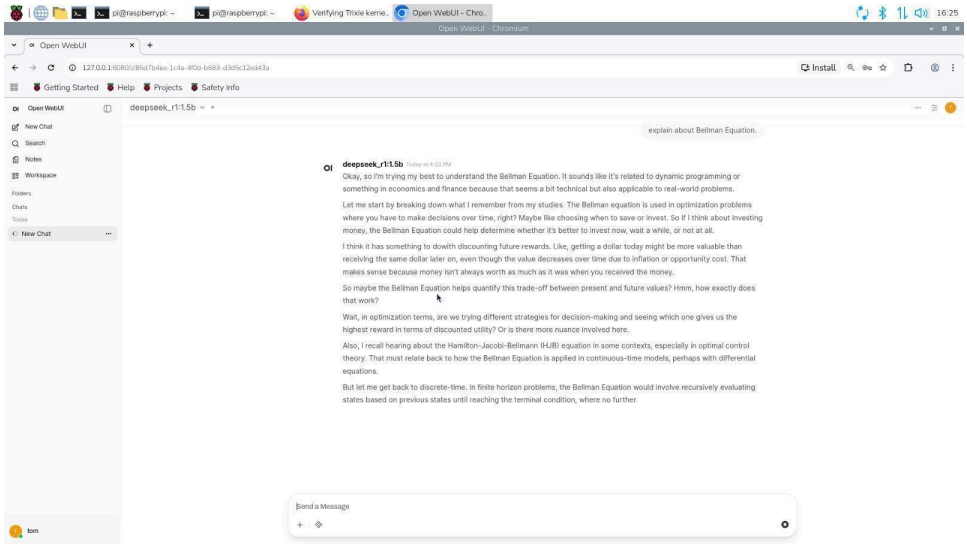
Live Inference Results

Once setup is complete, the Open WebUI interface is accessible from any browser on the same network. The screenshots below show the system running each model locally on the Hailo-10H — no internet connection is required during inference.

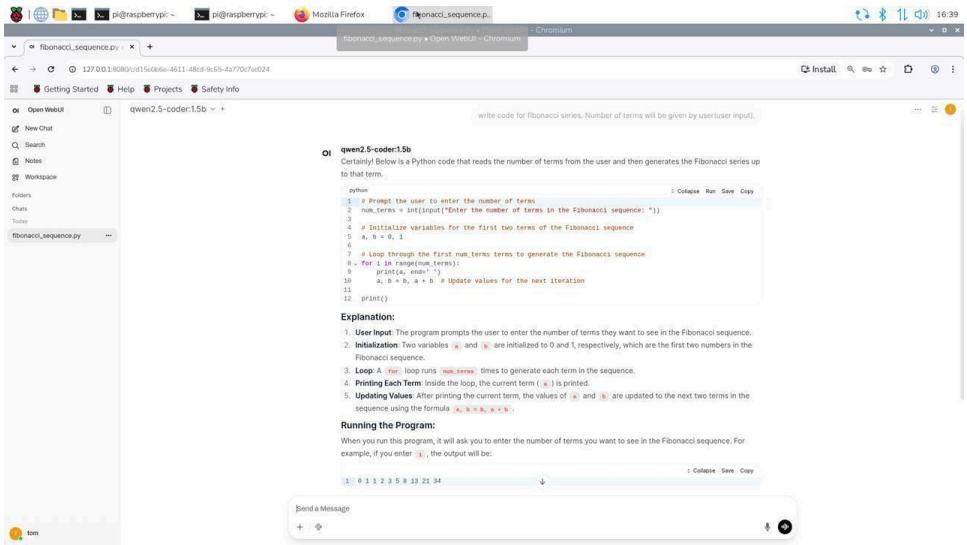
Select any model from the downloaded list to start a conversation. Only models that have already been pulled via the `hailo-ollama` API will appear as available options.



The screenshot below shows DeepSeek-R1-Distil generating a response directly on the device. The model uses chain-of-thought reasoning, which means it works through intermediate steps internally before producing the final answer.



The screenshot below shows Qwen2.5-Coder generating code on the device. All computation runs on the Hailo-10H via PCIe — the Raspberry Pi CPU is not involved in inference.



References

This document draws on the following sources. All descriptions and explanations have been written independently. References are included for verification and to credit original authors.

[1] *Raspberry Pi AI HAT+ 2 — Official Hardware Documentation —* https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html

[2] *Raspberry Pi AI Software Documentation (LLMs on AI HAT+ 2)* — https://www.raspberrypi.com/documentation/computers/ai.html

[3] *CNX Software — Raspberry Pi AI HAT+ 2 Review (LLM inference benchmarks)* — https://www.cnx-software.com/2026/01/20/raspberry-pi-ai-hat-2-review-a-40-tops-ai-accelerator-tested-with-computer-vision-llm-and-vlm-workloads/

[4] *Open WebUI — Official GitHub Repository* — https://github.com/open-webui/open-webui

[5] *Hailo Developer Zone — Software Downloads (HailoRT 5.2.0)* — https://developer.hailo.ai/developer-zone/sw-downloads/

[6] *Hailo — Bringing Generative AI to the Edge: LLM on Hailo-10H* — https://hailo.ai/blog/bringing-generative-ai-to-the-edge-llm-on-hailo-10h/

This document is open-source lab documentation. Corrections and additions are welcome via pull request.

(Updated on: 19-07-2026)